

第22回 実践：Web ゲームプログラミング1（ゲームループ）

実習時のフォルダ構成

Web_名前 の中に 22 フォルダを作る。22 の中に `sample` と `exercise` を作り、次のファイルを用意する。

```
Web_名前
├── 22
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- `sample`：講義で使うサンプル。今回から第 28 回まで、この `sample` に少しずつ書き足して、シューティングゲームを作っていく。
- `exercise`：自分のシューティングゲーム。今回から、講義と同じ機能を、自分のゲームにも追加していく。

`sample/index.html` と `exercise/index.html` には、どちらも次の内容を書く。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>シューティングゲーム</title>
</head>
<body>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

講義内容

以降の講義内容のコードは、`sample/main.js` に、上から順に書く。書いたら `sample/index.html` をブラウザで開き、画面で結果を確認する。

1. 前回の復習

- `ctx.fillStyle` で色を決め、`ctx.fillRect(x, y, 幅, 高さ)` で四角形を描く。
- `ctx.fillText("文字", x, y)` で文字を描く。

- 座標は、左上が (0, 0)。x は右へ、y は下へ行くほど大きい。

前は、絵を 1 回描くだけだった。今回は、絵を何回も描き直して、自機を動かす。

2. アニメーションの仕組み

パラパラ漫画のように、「少しだけ動かす → 描き直す」を素早く繰り返すと、動いて見える。

少し動かす（更新） → 描く（描画） → 少し動かす → 描く → …

- この「更新 → 描画」の 1 回分を、**1 フレーム**という。
- ブラウザでは、1 秒間に約 60 フレーム（60 回）繰り返す。
- この繰り返しを「ゲームループ」という。

3. 準備と変数

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
```

- `// ===== 準備 =====` のような見出しのコメントで、ファイルの中を役割ごとに分けておく。これから書き足す場所が分かりやすくなる。
- `frameCount` : 何フレーム目かを数える変数。1 フレームごとに 1 増やす。

4. 更新（動かす）

```
// ===== 更新 =====
function updatePlayer() {
  player.x += player.speed;
  // 右端から出たら、左端に戻す
  if (player.x > canvas.width) {
    player.x = -player.w;
  }
}

function update() {
  frameCount += 1;
  updatePlayer();
}
```

- `updatePlayer` : 自機を、1 フレームで `speed` (5) だけ右へ動かす。1 秒では約 300 (5 × 60) 動く。
- 右端から完全に出たら (`player.x` が `canvas.width` より大きくなったら)、左端の外 (`-player.w`) に戻す。

- `update` : 1 フレームで行う「更新」を、まとめて呼び出す関数。

5. 描画（描く）

```
// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function drawUI() {
  ctx.fillStyle = "#ffffff";
  ctx.font = "20px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText(`FRAME: ${frameCount}`, 10, 30);
}

function draw() {
  drawBackground();
  drawPlayer();
  drawUI();
}
```

- `draw` : 1 フレームで行う「描画」を、まとめて呼び出す関数。背景 → 自機 → 文字の順に描く。
- `drawUI` : 画面に文字（UI）を描く。今は、フレーム数を表示している。
- 毎フレーム、最初に背景でキャンバス全体を塗りつぶして、前のフレームの絵を消してから描き直す。

6. ゲームループ

```
// ===== ゲームループ =====
function gameLoop() {
  update();
  draw();
  requestAnimationFrame(gameLoop);
}

gameLoop();
```

- `requestAnimationFrame(gameLoop)` : 「次の画面の書き換えのタイミングで、`gameLoop` を呼び出してください」とブラウザに頼む。
- `gameLoop` の最後で、毎回これを頼むので、`gameLoop` が約 60 回／秒で繰り返し呼び出される。
- `()` の中には、関数の名前だけを書く（`gameLoop()` のように `()` を付けない）。`()` を付けると、その場で呼び出してしまう。
- 最後の `gameLoop()` ; で、最初の 1 回を呼び出して、ゲームループを始める。

7. 背景を描かないとどうなるか

`draw` の中の `drawBackground()`; の行の先頭に `//` を付けて、コメントにしてみる。

- 前のフレームの絵が消えずに残り、自機が線のように伸びて見える。
- 確認したら、`//` を消して元に戻す。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて確認する。今回から、`exercise` は自分のシューティングゲームとして作っていく。

演習 1 ゲームループを作る

- 講義と同じ形で、準備・自機・更新・描画・ゲームループを書き、自機が右へ動き続けるようにする。
- 自機の大きさ・速さ・色と、背景の色は、自分で決める。

演習 2 左へ動かす

- `updatePlayer` を書き換えて、自機が左へ動き続けるようにする。
- 自機が左端から完全に出たら (`player.x + player.w` が 0 より小さくなったら)、右端 (`canvas.width`) に戻す。

演習 3 経過時間を表示する

- `drawUI` に 1 行追加して、フレーム数の下に「TIME: ○」と、経過した秒数を表示する。
- 秒数は、フレーム数を 60 で割って、小数点以下を切り捨てて求める (`Math.floor()` を使う)。

発展 (任意)

- 30 フレームごとに、自機の色が 2 つの色で交互に切り替わるようにする。(ヒント: `Math.floor(frameCount / 30) % 2` が 0 か 1 かで、`fillStyle` を変える。)